

Introduction to Structured Data

WEEK 1

FINS FINANCIAL MARKET DATA

FINS5545

LEARNING OBJECTIVES

By the end of this lecture, you will be able to:

- Describe the four stages of the **Data Factory Floor** (DFF) and what each one produces
- Classify a financial dataset by its generator (fundamental, market, generated, alternative) and its structure (structured, semi-structured, unstructured)
- Choose between a list, a dict, a numpy array, and a pandas DataFrame for a given task
- Explain why Parquet is the default storage format for large tabular data, and when a CSV is still the right answer
- Query a CSV or Parquet file with DuckDB without loading it into pandas first
- Distinguish **panel**, **time-series**, and **cross-sectional** data, and slice between them with pandas
- Apply the full pipeline (load → clean → save → query → slice → plot) to a real daily-price panel in the workshop

REQUIRED READING

Course-mandated textbook

Hric and Lin (2026). *Applied Data Science in FinTech: Models, Tools, and Case Studies*.

This week, read:

- **Section 1** – Data Science for FinTech
- **Chapter 1** – Data Science in FinTech
- **Chapter 2** – Data Management for FinTech

These lecture slides and the in-house chapter are *companion* material to the textbook rather than a replacement for it, and they apply the same tools the textbook teaches to the dataset you analyse in the workshop.

WHY THIS COURSE

A recurring scenario in most finance roles:

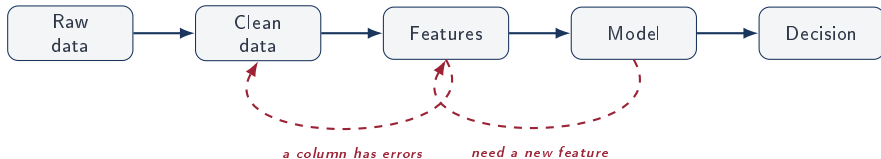
- A decision needs to be made: price an asset, size a trading position, meet a capital requirement, answer a client's question
- The decision rests on a number, the number rests on data, and that data often contains errors or arrives in the wrong format
- Most of the work is cleaning and formatting that data so it is ready for modelling

raw data → clean data → features → model → decision

Week 1 covers the first arrow: turning raw, messy data into a clean, structured table.

THE DFF PROCESS LOOPS BACK TO EARLIER STAGES

The work rarely runs straight through. Building features often reveals errors the cleaning step missed, and modelling often needs a feature that has not been built yet.



- Forward arrows trace the standard sequence: raw → clean → features → model → decision
- Dashed back-arrows show the two common corrections: a feature step finds errors in a column, or a model step needs a feature that does not yet exist
- Returning to an earlier stage is routine: we correct or augment the data there, then move forward again

Later stages routinely return work to earlier ones, so the pipeline is best treated as a loop rather than a straight line.

WHAT WEEK 1 COVERS

This week works through seven topics, then a workshop:

1. **What Is Structured Data** – data in rows and columns, the layout pandas and SQL expect
2. **The Data Factory Floor** – the stages data moves through, from raw to decision
3. **Data Types** – the four kinds of financial data, and what each is good for
4. **Python Data Structures** – how data is held in memory (list, dict, array, DataFrame)
5. **Data Formats** – how data is stored on disk (CSV, Parquet, JSON)
6. **SQL and DuckDB** – querying files as if they were a database
7. **Panel, Time-Series, Cross-Section** – the three layouts of structured data

The **workshop** at the end applies all seven to a real daily-price panel.

WHAT IS STRUCTURED DATA?

STRUCTURED DATA: ROWS, COLUMNS, TYPES

Structured data fits in a table where:

- Each **row** is one observation (a trade, a company-day, a rating action)
- Each **column** is one variable (Price, Volume, Ticker)
- Each **cell** has a known **type** (float, integer, date, string)
- The **schema** (the column names and types) is fixed across rows

This is the form database textbooks call a *relation*. The framework was set out by Codd (1970) at IBM Research and underpins the major relational databases (Oracle, Postgres, SQL Server) and the DataFrame libraries (pandas, R, Stata) in use today.

STRUCTURED, SEMI-STRUCTURED, AND UNSTRUCTURED DATA

Structured (table)

Date	Tic	Price
2026-04-14	AAPL	170.4
2026-04-14	MSFT	415.1
2026-04-15	AAPL	172.0
2026-04-15	MSFT	417.6

rows $\times$ columns, one data type per column

Semi-structured (JSON)

```
{  
  "ticker": "AAPL",  
  "trades": [  
    {"t": "09:30", "px": 170.4},  
    {"t": "09:31", "px": 170.6}  
  ],  
  "splits": []  
}
```

nested, schema can vary row-to-row

Unstructured

"Risks. Inflation, supply-chain disruption, and FX exposure remain material to our 2026 outlook ..."

Text, audio, or image, with no schema at all.

free-form, requires natural-language processing (NLP) or computer vision to extract a number

TOOLS EXPECT A STRUCTURED TABLE

Tools like pandas, SQL, and Excel are built to read a table whose columns each have a declared data type. That fixed layout is what makes them work:

- **Types are known:** each column holds one type — Price is a float64 — so a tool reads it without guessing on every row
- **Operations work the same way:** filtering, grouping, joining, and aggregating are defined on tables and chain one after another
- **Major tools expect it:** pandas, SQL, Excel, R, Stata, and most plotting libraries take a table as input
- **Storage is compact:** Parquet stores each column together (CSV stores each row together) and compresses it, so the file is much smaller than the same data as CSV (Part E compares them)
- **Raw input ends up here:** even when the source is JSON, HTML, or a PDF, the cleaning stage turns it into a structured table

pandas, SQL, and most plotting tools expect a structured table, so the pipeline's first job is to convert raw inputs into that form.

THE DATA FACTORY FLOOR

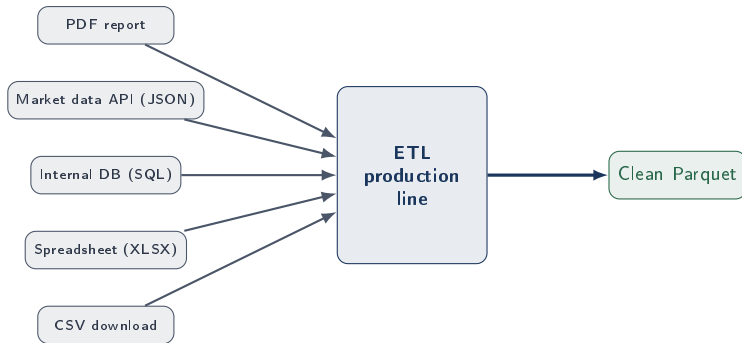
WHY WE TREAT THE WORKFLOW AS A FACTORY

Financial data rarely arrives ready to use:

- Prices in one file, corporate actions in another, currencies in a third
- Missing rows on public holidays, duplicate trades from exchange feeds
- "N/A" stored as a string. Dates written 14/04/2026 in one locale and 04/14/2026 in another
- A data vendor can add, rename, or drop a column without warning, breaking code that expected the old layout

Turning that into a reliable input for a model needs a *production line*: the same steps, in the same order, every time. That line is the **Data Factory Floor** (DFF), the framework laid out by Hric and Lin (2026, Chapter 2 – required reading).

DATA SOURCES AND THE ETL STAGE



- Each source has its own file format, its own ticker codes for the same security, and its own update frequency
- Stage 1 (ETL) handles *all five at once*, so downstream code reads a single structured table instead of five different formats
- Clean Parquet is one common output, but a CSV, a SQL table, or another columnar format all work — the choice is downstream of the ETL contract

THE FOUR STAGES OF THE DFF

Each stage has an output and a single **critical attribute** – the property that, if absent, means the work is not fit for the next stage to use (Hric and Lin, 2026).

1. **Stage 1: Data Lake (ETL)** – extract, transform, load. Stage 1 produces the single source of truth that every later stage depends on, so any error it passes through corrupts the stages after it. *Output: a clean, structured table. Critical attribute: **authenticity** – the data faithfully represents its source.*
2. **Stage 2: Prompt and Feature Engineering** – turn clean data into model inputs (returns, volatility, momentum, sentiment scores). *Output: a feature matrix. Critical attribute: **utility**.*
3. **Stage 3: Model Design** – fit, validate, and compare candidate models using the features from Stage 2. *Output: a trained model with performance estimates. Critical attribute: **clarity and parsimony**.*
4. **Stage 4: Implementation** – deliver to a dashboard, a trading system, a report, or an API. *Output: a number a decision-maker acts on. Critical attribute: **added value**.*

STAGE 1: DATA LAKE (ETL) IN DETAIL

- **Extract:** download the raw data. Examples: request end-of-day prices from a financial market data API, query the Fed FRED database for interest rates, download a 10-K filing from SEC EDGAR
- **Transform:** convert the raw data into consistent, analysis-ready columns – parse dates, adjust for splits and dividends, remove duplicate trades, standardise tickers, correct dtypes, and fill or drop missing values
- **Load:** save the cleaned data where it can be queried – a Parquet file on local disk at student scale, or a warehouse (Snowflake, BigQuery) or data lake in industry
- Stage 1 produces the single source of truth and is where the **GDPR** (data-protection) and **PSD2** (payments) rules are enforced – every later stage depends on the data it passes through

Practitioner surveys (e.g., Anaconda's annual State of Data Science) routinely find that data scientists spend more time on data cleaning and preparation than on any other task – typically a majority of the working week. Treat precise percentages with caution: the surveys differ in scope and method.

STAGES 2, 3, AND 4: OVERVIEW

An analyst building a momentum trading signal across a portfolio of equities moves through Stages 2 to 4:

- **Stage 2 (Features)**: turn the cleaned price series into a 20-day rolling momentum column, a 60-day rolling volatility column, and a cross-sectional Z-score. *Tools: pandas, numpy.*
- **Stage 3 (Model)**: regress next-month returns on today's momentum with train/test splits to estimate signal strength. *Tools: statsmodels, scikit-learn.*
- **Stage 4 (Implementation)**: emit a daily CSV of ranked signals, push it to the portfolio manager's dashboard, monitor for input-data drift. *Tools: Streamlit, SQL, a scheduler.*

Week 1 focuses on Stage 1 – the input data the later stages build on.

THE HAND-OFFS: THE MOST COMMON SOURCE OF BUGS

Each stage has an **input**, an **output**, and a **hand-off**. A broken hand-off is the most common cause of wrong results downstream – here one bad value cascades through all four stages:

- **Stage 1** → **Stage 2**: a single row arrives as "N/A", so one price column silently changes dtype from `float64` to `object`
- **Stage 2** → **Stage 3**: the rolling mean skips the bad rows silently and passes on a shorter series
- **Stage 3** → **Stage 4**: the model trains on the shortened series and reports a strong backtest
- **Stage 4 (live)**: the live feed has no "N/A" values, so the real signal behaves nothing like the backtest

Industry calls the fix a **data contract**: an explicit schema, value ranges, and update frequency pinned to the dataset so the break is caught at the hand-off (Rohrer, 2024).

DATA TYPES BY GENERATOR

WHO GENERATES FINANCIAL DATA?

Who produces a dataset tells you a lot about it: how often it updates, how accurate it is, what biases it carries, and what it costs. Financial data falls into four groups by producer:

1. **Fundamental** – governments and companies. *Examples: GDP, CPI, 10-K filings, quarterly earnings, FOMC minutes.*
2. **Market** – exchanges and central banks. *Examples: NYSE and NASDAQ trade prices, order books, Treasury yields, Federal Reserve policy rates.*
3. **Generated** – analysts, researchers, machines. *Examples: sell-side earnings forecasts, credit ratings, NLP-derived sentiment scores.*
4. **Alternative** – the web, sensors, devices. *Examples: satellite car-park counts, credit-card panels, Reddit sentiment, job-posting volumes.*

FUNDAMENTAL DATA

Data about the *underlying* economic entity, not its traded price.

- **Governments:** macro indicators (GDP, CPI, unemployment), FOMC minutes, Fed balance-sheet releases, Treasury auction results
- **Companies:** annual reports (10-K), quarterly filings (10-Q), earnings-call transcripts, insider transactions, dividend declarations
- **Update frequency:** low. Most updates are quarterly or annual because filings are audited
- **Form:** tabular in headline numbers such as revenue, earnings per share (EPS), and debt, and free text in the management discussion (MD&A) and risk-factor sections
- **Typical sources:** SEC EDGAR, FRED, BEA, company investor-relations pages
- **Example:** a large-cap 10-K often runs to several hundred pages, though length varies widely – yet its structured core is tiny, with the income statement only a handful of rows and the risk-factor section often twenty pages or more

MARKET DATA

Data about *trading activity*: prices, quotes, volumes, order books.

- **Transactional**: every trade and quote update is recorded. A single major equity exchange's market-data feed runs into the **tens of billions** of messages per day across trades, quotes, and order updates
- **Reference**: interest-rate curves, FX fixings, index constituents, corporate-action calendars
- **Update frequency**: faster than any other data in finance. Microsecond timestamps on equities, sub-microsecond on FX venues
- **Vendors**: Bloomberg, Refinitiv, FactSet (professional); WRDS (Wharton Research Data Services), CRSP, and TAQ trade-and-quote data (academic); Polygon, Alpaca, yfinance (retail and student)
- **Form**: almost always structured. Rows and columns with well-defined types

Daily closing prices, like the dataset we meet in the workshop, are the coarsest slice of an otherwise tick-by-tick feed.

Human or algorithmic *interpretation* of other data – a derived view rather than a fresh measurement.

- **Sell-side research:** an analyst at a major bank publishing a price target on a stock, or consensus EPS forecasts from estimate databases such as IBES or FactSet
- **Credit ratings:** S&P, Moody's, Fitch letter grades and the written reports behind them
- **Machine-derived:** Ravenpack sentiment scores from news, Bloomberg machine-readable news flags, LLM-tagged earnings-call topic labels
- **Why it is useful:** it compresses many raw inputs into one decision-ready number
- **Why to be careful:** it embeds the generator's biases and may revise history (analysts change targets, ratings get upgraded)

ALTERNATIVE DATA

Everything that was not originally collected for financial analysis but turns out to predict something.

- **Satellite**: car-park counts at retail chains predict quarterly revenue surprises (Orbital Insight)
- **Card panels**: anonymised credit-card transactions track consumer spending in near real-time (Second Measure, Earnest)
- **Web scrape**: job-posting volumes flag hiring slowdowns before they reach earnings (Thinknum). Online price panels feed inflation nowcasts (PriceStats, MIT)
- **Social and text**: Reddit WallStreetBets sentiment flagged the GameStop squeeze days before it happened in January 2021
- **Industry size**: hedge-fund and asset-manager spending on alternative data is tracked by industry sources (e.g., AlternativeData.org, Eagle Alpha) and runs into the multiple billions of US dollars per year, growing rapidly

PYTHON DATA STRUCTURES

FOUR PYTHON DATA STRUCTURES

How is a structured table held *in memory* while you work on it?

1. **list** – ordered, mutable, any mix of types. *Use when order matters and types can vary.*
2. **dict** – key-to-value lookup that stays fast as the dict grows. *Use when you want to look up by name, not position.*
3. **numpy array** – one homogeneous type, fixed size, vectorised numerical maths in C. *Use for tight numeric loops and matrix maths.*
4. **pandas DataFrame** – labelled rows and columns, mixed types per column, rich API. *Use for anything that looks like a spreadsheet.*

The DataFrame is the default choice for tabular financial data. It is built on the numpy array, while the list and dict are simpler containers for small or non-tabular data.

CODE: LIST AND DICT

```
# Four daily closing prices for a hypothetical stock

# list: ordered, indexed by position
closing_prices = [100.0, 101.3, 100.7, 102.1]
closing_prices[0]          # 100.0

# dict: indexed by key (the date itself)
price_by_date = {
    "2026-04-14": 100.0, "2026-04-15": 101.3,
    "2026-04-16": 100.7, "2026-04-17": 102.1,
}
price_by_date["2026-04-15"]    # 101.3
```

- **list**: ordered and indexed by position, can mix types, but arithmetic needs a slow Python loop
- **dict**: access by key stays fast even as the dict grows, and keys have no inherent order
- **Choose by access pattern**: “what is the i -th item?” → list. “what is the value for this date?” → dict

CODE: NUMPY ARRAY AND DATAFRAME

```
import numpy as np, pandas as pd

# numpy array: one column of numbers, C-fast maths
prices_array = np.array([100.0, 101.3, 100.7, 102.1])
prices_array.mean()           # 101.025

# pandas DataFrame: rows x columns, labels + types
prices_df = pd.DataFrame({
    "date": pd.to_datetime(list(price_by_date)),
    "close": list(price_by_date.values()),
})
prices_df["close"].pct_change() # daily price returns
```

- **numpy array**: one type per array, kept together in memory, arithmetic in compiled C
- **DataFrame**: many columns (each one type), one shared row index, like a group of Series
- **Nesting**: a DataFrame column *is* a Series, and a Series wraps a numpy array, so the DataFrame gets numpy's speed and adds row and column labels on top

WHEN TO USE EACH DATA STRUCTURE

- **list**: a short, ordered sequence, items can be of mixed type (e.g. the tickers you loop over)
- **dict**: one-row records, named lookups, JSON parsing, config loaded from a YAML file (a plain-text settings format)
- **numpy array**: vectorised maths on a single type (a return vector, a covariance matrix, a single row of model features)
- **DataFrame**: anything tabular with named columns and mixed types per column – the default for financial data

When in doubt with tabular financial data, use a DataFrame. The lighter-weight structures are worth reaching for only when a DataFrame would be excessive.

WHAT A DATAFRAME IS MADE OF

- A DataFrame behaves like an ordered dict: keys are column names, values are `pandas.Series` objects
- A Series is a numpy array with an attached index (the row labels)
- A numpy array holds one data type in a single block of memory, with fixed dimensions
- Nesting: **numpy array** $\subset$ **Series** $\subset$ **DataFrame**
- A Python for loop over DataFrame rows is slow. Vectorised column operations or `groupby` do the same work far faster (McKinney, 2022)

In practice you do almost all your work at the DataFrame level. The nesting matters because the numpy arrays underneath are what make vectorised column operations and `groupby` fast — far faster than looping over rows.

DATA FORMATS ON DISK

THREE COMMON FILE FORMATS

- **CSV** – comma-separated values. Plain text and human-readable, but stores no type information — every value is text until you parse it
- **Plain text / JSON** – free-form text or tag-structured records. Flexible, but requires parsing before analysis
- **Parquet** – compressed binary, *columnar* (it keeps each column's values together, not each row's), and records every column's type in the file. Built for fast column reads over large tables (Apache Software Foundation, 2024)

CSV is the slowest to load but the most portable, JSON is the most flexible, and Parquet is the fastest for analysis.

CSV (COMMA-SEPARATED VALUES)

```
Date,Ticker,Price,Volume
2026-04-14,XYZ,112.45,3214500
2026-04-15,XYZ,113.10,3541200
2026-04-16,XYZ, N/A,1123400
```

- Readable by any tool, language, or operating system
- **No types:** Price loads as object (string) because of the "N/A", so you must coerce it back to numeric
- **No compression:** a one-million-row price table is often 100+ MB
- **No schema:** next quarter's file could rename Ticker to Symbol and nothing would warn you
- Right for: sharing with a non-technical colleague, tiny files, Excel handoffs

PARQUET

- **Columnar storage:** Parquet keeps all of Price together, then all of Volume — unlike CSV, which stores each row's fields together — so a query that needs two columns reads only those two
- **Column types are stored:** Price stays float64, Date stays datetime64 — no re-parsing on load
- **Compressed:** Snappy or ZSTD compression shrinks financial data by roughly **2–10x**, more when columns contain many repeated values
- **Skips blocks that can't match:** Parquet stores each column's min and max per block of rows, so a filter like `Date >= '2026-01-01'` reads only the blocks that might contain matches
- **Open source:** maintained by the Apache Software Foundation, supported by pandas, DuckDB, Spark, Polars, and R

Why this is fast: most queries read only a few columns and aggregate them, so reading just those columns skips most of the file.

WHEN TO USE EACH FORMAT

- Sharing with a human or a spreadsheet tool: **CSV**
- Storing for reuse, a research pipeline, or anything over 1 MB: **Parquet**
- Querying a file too large to fit in memory: **DuckDB** over Parquet — it streams through the file in chunks instead of loading all of it
- Raw API responses, logs, and anything text-based: **plain text / JSON**, kept until ETL turns it structured
- Crossing a team boundary where the schema matters: Parquet, paired with a data-contract document

Save your own working data as Parquet, and export to CSV only when someone else asks for it.

SQL AND DUCKDB

You are not examined on this content – but it is essential knowledge for working with large financial data.

- **SQL** is the shared language of tabular data. Most finance roles use SQL at some point
- **Declarative**: you describe the result you want, and the database works out how to produce it. In Python you usually do the opposite — you write the steps that compute the result yourself
- Five SQL keywords cover most of what you write: `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `JOIN`
- Core idea: rows in, rows out, described by set operations on tables
- Sits above file formats, databases, and warehouses alike: once you know SQL, the backing store becomes a detail

MINIMUM VIABLE SQL

```
SELECT Ticker ,  
       AVG(Price)      AS mean_price ,  
       COUNT(*)        AS n_days  
FROM   prices  
WHERE  Date >= '2026-01-01'  
GROUP BY Ticker  
ORDER BY mean_price DESC  
LIMIT 10;
```

- Reads as: *from the prices table, keep rows dated 2026 or later, group by ticker, compute the average closing price and a count of days, return the top ten tickers by mean price*
- **Declarative:** you describe the result, not the steps to compute it. The database plans how to run the query
- **Five-keyword pattern:** SELECT, FROM, WHERE, GROUP BY, ORDER BY (with JOIN added when more than one table is involved) covers most of what you write

DUCKDB: SQL QUERIES OVER LOCAL FILES

- **DuckDB** is an in-process columnar database (Raasveldt and Mühleisen, 2019)
- **No server:** `pip install duckdb` and call it from Python. It runs inside your process like SQLite, but with a columnar engine built for fast reads and aggregations over large tables
- **Reads files directly:** point it at a CSV, Parquet, or JSON path and query
- Released in 2019 by Mark Raasveldt and Hannes Mühleisen at CWI Amsterdam, and widely adopted for local data analysis by 2024
- A quick way to run SQL on files you already have, with no separate CREATE TABLE or data-load step

QUERYING A PARQUET FILE WITH DUCKDB

```
import duckdb
top = duckdb.sql("""
    SELECT Ticker, AVG(Price) AS mean_price
    FROM 'prices.parquet'
    WHERE Date >= '2026-01-01'
    GROUP BY Ticker ORDER BY mean_price DESC LIMIT 10
""").df()
```

- Same query as the previous slide, run against a Parquet file
- The file is the table: no CREATE TABLE, COPY, or load step
- .df() returns a pandas DataFrame, so the result is ready for further pandas work
- Swap prices.parquet for prices.csv and the same code runs

WHEN TO USE SQL

- **Combining tables:** joining several tables in one SQL query is easier to read than the same chain of `pd.merge` calls in pandas
- **Aggregating a file too big for memory:** DuckDB streams through the Parquet without loading all of it
- **Expressing business logic:** SQL uses plain, English-like keywords, so another analyst can often read a query without extra explanation
- **Repeatable reports:** a saved query re-runs to regenerate the same report each time the data updates

Parquet stores each column's type in the file, so DuckDB can query it directly off disk without a separate load step.

PANEL, TIME-SERIES, CROSS-SECTION DATA

THE TIDY PRINCIPLE

Throughout this section the example is a daily stock-price panel: one row per stock per date, with a *Ticker* naming the stock and measured columns such as Price, Return, and Volume.

Following Wickham (2014):

1. Each **variable** forms a column — Price, Return, and Volume each become their own column
2. Each **observation** forms a row (one row per *Ticker-Date* pair, not one row per ticker with dates spread across columns)
3. Each **kind of thing you observe** gets its own table (prices in one table, corporate actions in another)

Tidy data looks simple in retrospect, but arranging non-tidy data into that form is where the bulk of cleaning time goes, because most downstream tools (groupby, merge, plotting libraries) assume tidy input.

THE QUESTION DETERMINES THE LAYOUT

The layout you need — time-series, cross-section, or the full panel — depends on the question you ask:

Question	Layout
“How has Apple’s stock performed over the last 26 years?”	time-series
“How did the Tech 4 stocks react on the COVID-crash Monday (16 Mar 2020)?”	cross-section
“Across all US stocks since 2000, do high-momentum stocks earn higher returns?”	panel

A time-series follows one stock through time, a cross-section compares many stocks on a single date, and a panel holds both dimensions at once.

SLICING A PANEL TO TIME-SERIES OR CROSS-SECTION

A toy monthly stock panel: each row is a stock (*Ticker* A, B, C), each column a month (t_1 – t_4), and each cell that stock's Price in that month. The three views below read different parts of this table: the whole grid, a single row, or a single column.

Panel

everything: 3×4

	t_1	t_2	t_3	t_4
A	100	101	102	103
B	50	51	52	53
C	75	74	76	77

12 observations *the full panel*

Time-series

filter to ticker A

	t_1	t_2	t_3	t_4
A	100	101	102	103
B	50	51	52	53
C	75	74	76	77

1 row, 4 dates
how A changed

Cross-section

filter to date t_2

	t_1	t_2	t_3	t_4
A	100	101	102	103
B	50	51	52	53
C	75	74	76	77

3 rows, 1 date
snapshot at t_2

THE TIME-SERIES: AAPL, 26 YEARS (REAL DATA)

Time-series filter: one ticker, all dates.

- Tech 4 example: AAPL alone = **6,539** rows
- Question template: “How did X change?”
- Tools: rolling, expanding, resample, shift
- Source: the workshop panel, filtered to `Ticker == "AAPL"`

Real AAPL endpoints from the workshop panel:

Date	Tic	Price	TotalReturn
2000-01-03	AAPL	111.94	0.0888
2000-01-04	AAPL	102.50	-0.0843
⋮	⋮	⋮	⋮
2025-12-30	AAPL	273.08	-0.0025
2025-12-31	AAPL	271.86	-0.0045

One row per trading day, sorted by Date: the full daily price history of a single company.

THE CROSS-SECTION: COVID-CRASH MONDAY (REAL DATA)

Cross-section filter: all tickers, one date.

- Tech 4 example: 16 March 2020 = 4 rows
- Question template: “Which entity performed best at moment t ?”
- Tools: `rank`, `.corr()`, regression of returns on characteristics

The COVID-crash Monday (16 Mar 2020), sorted worst to best:

Date	Tic	Price	TotalReturn
2020-03-16	NVDA	196.40	−18.5%
2020-03-16	MSFT	135.42	−14.7%
2020-03-16	AAPL	242.21	−12.9%
2020-03-16	ORCL	42.72	−10.9%

Same date, four tickers ranked: the market on a single day.

CONVERTING BETWEEN THE LAYOUTS

From	To	Operation
Panel	Time-series	Filter to one entity: <code>panel[panel["Ticker"]=="A"]</code>
Panel	Cross-section	Filter to one date: <code>panel[panel["Date"]=="2026-01-03"]</code>
Panel	Wide	<code>panel.pivot(index="Date", columns="Ticker", values="Price")</code>
Wide	Panel (long)	<code>wide.reset_index().melt(id_vars="Date", ...)</code>
Many time-series	Panel	<code>pd.concat([ts_A, ts_B, ts_C])</code>

- Panel is the standard storage layout. Keep data in long-form panel until you need something else
- Wide form is the same panel reshaped into a $\text{Date} \times \text{Ticker}$ grid, useful for return-matrix maths
- Most research methods in this course operate on one of these four layouts

WIDE VS LONG

The same nine numbers in two different layouts.

Wide

one column per ticker, good for matrix maths

Date	A	B	C
2026-01-02	100	50	75
2026-01-03	101	51	74
2026-01-04	102	50	76

Long / Panel

one row per (Ticker, Date), good for groupby and merge

Date	Ticker	Price
2026-01-02	A	100
2026-01-02	B	50
2026-01-02	C	75
...	...	...

Store data in long form, and pivot to wide only for the specific calculation that needs an entity $\times$ date matrix.

RESHAPING BETWEEN LONG AND WIDE: MELT AND PIVOT

```
# long panel -> wide (one column per ticker) -- a transient view
wide = panel.pivot(
    index="Date", columns="Ticker", values="Price"
)

# wide -> long panel -- the storage layout
long_again = wide.reset_index().melt(
    id_vars="Date", var_name="Ticker", value_name="Price"
)
```

- pivot: panel $\rightarrow$ wide. Create the wide form only for an operation that needs a rectangular matrix — a correlation matrix, or a return matrix on a small set of tickers
- melt: wide $\rightarrow$ panel. Converts the temporary wide form back to long after the calculation is done
- These are reshapes of the same data, but *they are not equally good ways to store it*

STORING A PANEL IN LONG FORM

Finance panels are stored in long form, not wide. You reshape to wide only for a single calculation, then go back. The reasons are stronger in finance than elsewhere (Wickham, 2014):

- **Entries and exits.** New tickers appear at IPOs, others are removed at delistings, and some are renamed in mergers. CRSP holds **>30,000** historical US tickers, but at any point in time only a few thousand are listed. A wide (Date $\times$ Ticker) matrix over the full span is mostly NaN – not because listed stocks fail to trade, but because most stocks did not yet exist or no longer exist on most dates.
- **Many variables.** A real panel carries Price, Volume, Return, Volatility, BookValue, Spread, ... Long adds variables as columns. Wide forces (variables $\times$ tickers) columns or one file per variable.
- **Append-friendly.** Each new trading day is N new rows in long form. Wide requires a schema migration whenever a ticker IPOs or delists.
- **Engine-friendly.** pandas, DuckDB, Parquet, and most SQL warehouses: optimised for many-rows-few-columns. PostgreSQL caps at **1,600** columns, BigQuery at **10,000**.
- **Industry standard.** CRSP, Compustat, TRACE, and most academic panel-data products are distributed in long form.

Pivot to wide only for the one calculation that needs an entity $\times$ date matrix, then go back to long — never persist wide form to disk.

WORKSHOP: THE TECH 4 PANEL

The workshop applies the tools from this lecture to a real dataset:

- **Tickers:** AAPL (Apple), MSFT (Microsoft), NVDA (NVIDIA), ORCL (Oracle)
- **Frequency:** daily closing prices and returns
- **Span:** 2000-01-03 to 2025-12-31 (26 years of trading)
- **Size:** **26,159** rows total, roughly **6,540** per ticker
- **File:** fins2026/week1/data/week1_workshop_panel.csv (and .parquet)
- **Columns:** Date, CompanyName, Ticker, Price, TotalReturn, PriceReturn

The workshop runs all of this lecture's tools on this dataset: load, clean, store, query, slice, and plot.

WORKSHOP: THE SIX-STEP PIPELINE

Build a clean Tech 4 price panel, then query, slice, and plot it.

Six steps, each a tool from the lecture:

1. **Load** the workshop CSV with the correct date parser
2. **Clean** dtypes and re-save as `week1_workshop_panel.parquet`
3. **Query** the Parquet with DuckDB for a per-ticker summary
4. **Slice** the panel into a time-series (AAPL) and a cross-section (2020-03-16)
5. **Pivot** to wide and compute returns across tickers
6. **Plot** the growth of \$1 invested in each ticker on a log-y axis

In the snippets that follow, file paths are relative to the `fins-agent` repository root.

PANDAS: THE STANDARD TABULAR-DATA LIBRARY IN PYTHON

pandas is the Python library this course uses throughout. The history is unusually finance-relevant.

- **Origin.** Started in 2008 by Wes McKinney while at **AQR Capital Management**, a quantitative hedge fund. AQR needed a high-performance, R-style DataFrame for cleaning and analysing financial panel data, and Python had nothing equivalent.
- **The name.** “pandas” is short for *panel data*, the econometrics term for a dataset indexed by both entity and time. The layout we just defined (one row per Ticker–Date pair) is exactly what the library was built for.
- **Open-sourced** in 2009. The project moved to a community of maintainers and is now part of the **NumFOCUS** foundation. Wes McKinney went on to write *Python for Data Analysis* (McKinney, 2022), the standard reference, freely available online.
- **Today.** The most-downloaded Python data-analysis library by a wide margin. Used by most major banks, hedge funds, and regulators that run Python. Underpins almost all the other libraries in this course (DuckDB returns DataFrames, matplotlib reads them, numpy backs them).

pandas was built at AQR to clean the exact panel-data layout Week 1 introduces, which is why most operations in this course slot into its API without friction.

STEP 1-2: LOAD, CLEAN, RE-SAVE

```
import pandas as pd
panel = pd.read_csv(
    "fins2026/week1/data/week1_workshop_panel.csv",
    parse_dates=["Date"], dayfirst=True,    # 03/01/2000 -> 3 Jan
)
panel = panel.drop_duplicates(subset=["Date", "Ticker"])    # see next
                  slide
panel = panel.sort_values(["Ticker", "Date"]).reset_index(drop=True)
panel.to_parquet("fins2026/week1/data/week1_workshop_panel.parquet")
panel.shape          # (26156, 6)    -- three rows dropped
panel.dtypes["Date"] # datetime64[us]
```

- **dayfirst=True** fixes the DD/MM/YYYY trap: 03/01/2000 is 3 January, not 1 March
- **Dedup** on (Date, Ticker) drops three CRSP rows, then `sort_values` restores tidy order
- **Parquet save** preserves the `datetime64` dtype, so later loads need no `parse_dates`

DFF STAGE 1 IN ACTION: EVEN CRSP HAS DUPLICATES

This panel comes from **CRSP** (Center for Research in Security Prices), the standard academic source for US equity prices. Even so, it contains duplicate rows:

```
panel.duplicated(subset=["Date","Ticker"]).sum()      # 3
panel[panel.duplicated(subset=["Date","Ticker"], keep=False)]
```

#	<i>Date</i>	<i>Ticker</i>	<i>Price</i>	<i>TotalReturn</i>	
#	2004-11-15	MSFT	27.39	0.016683	(x2)
#	2012-12-12	ORCL	31.94	-0.006803	(x3)

Three identical rows in 26,159 are enough to break the next step: `panel.pivot(index="Date", columns="Ticker", ...)` raises `ValueError: Index contains duplicate entries the moment a single (Date, Ticker) pair repeats.`

This is exactly Stage 1 of the Data Factory Floor: clean data is a checked claim rather than an assumption, so probe the panel before you pivot it.

STEP 3: QUERY WITH DUCKDB

```
import duckdb
ranks = duckdb.sql("""
    SELECT Ticker,
           AVG(Price)           AS mean_price,
           STDDEV_POP(Price) AS sd_price,
           COUNT(*)             AS n_days
    FROM 'fins2026/week1/data/week1_workshop_panel.parquet'
    GROUP BY Ticker ORDER BY mean_price DESC
""").df()
#   Ticker  mean_price  sd_price  n_days
#   AAPL      177.44      144.62    6539  (NVDA, MSFT, ORCL follow)
```

- **File-as-table:** query the Parquet directly — no CREATE TABLE, COPY, or server
- **STDDEV_POP** is population SD (n), while pandas `.std()` defaults to sample SD ($n-1$) — match on ddof
- **Equal `n_days`** across all four tickers confirms the Step 2 dedup worked

STEP 3 (ALTERNATIVE): THE SAME THING IN PANDAS

Once panel is in memory, the same per-ticker summary is a groupby. Use `ddof=0` so the pandas standard deviation matches DuckDB's `STDDEV_POP` (pandas defaults to sample SD with `ddof=1`).

```

ranks = (
    panel.groupby("Ticker")["Price"]
        .agg(mean_price="mean",
             sd_price=lambda s: s.std(ddof=0),
             n_days="size")
        .sort_values("mean_price", ascending=False)
        .reset_index()
)
#      Ticker  mean_price  sd_price  n_days
# 0    AAPL      177.44      144.62    6539
# 1    NVDA      115.87      165.13    6539
# 2    MSFT      110.73      127.88    6539
# 3    ORCL       48.39       45.55    6539

```

- **Use SQL/DuckDB:** file-as-table, no load step, multi-file joins
- **Use pandas:** table already in memory, next step is more pandas
- Watch the `ddof` default: `.std()` alone gives sample SD and will not match `STDDEV_POP`

STEP 4: SLICE TO TIME-SERIES AND CROSS-SECTION

```
# Time-series: Apple alone, all dates
aapl_timeseries = panel[panel["Ticker"] == "AAPL"].sort_values("Date")
aapl_timeseries.shape                                # (6539, 6)

# Cross-section: all tickers on the COVID-crash Monday
cross_section = panel[panel["Date"] == "2020-03-16"]
cross_section[["Ticker", "Price", "TotalReturn"]]

#   Ticker   Price   TotalReturn
#   AAPL    242.21    -0.1286
#   MSFT    135.42    -0.1474
#   NVDA    196.40    -0.1849
#   ORCL     42.72    -0.1093
```

- Two filters on the same panel: one gives a time-series, the other a cross-section
- Time-series layout: many rows, one ticker. Cross-section layout: many tickers, one date
- Same DataFrame, two questions: “how did AAPL change over time?” vs “which ticker held up best on 16 March 2020?”

STEP 5-6: PIVOT TO RETURNS, COMPOUND, AND PLOT

```
wide_return = panel.pivot(  
    index="Date", columns="Ticker", values="TotalReturn"  
)  
.dropna()  
# Plot growth of $1 (not cumprod-1: that goes negative, and logy drops  
it)  
growth = (1 + wide_return).cumprod()  
growth.tail(1).round(2)  
# Ticker      AAPL      MSFT      NVDA      ORCL  
# 2025-12-31 352.68    13.53    2079.88    8.68  
  
growth.plot(title="Tech 4 growth of $1, 2000-2025",  
            logy=True, ylabel="value of $1 (log)")
```

- **Pivot** long to wide on TotalReturn — the right layout for column-wise return maths
- **\$1 grew** over 2000–2025 to **\$2,080** (NVDA), \$353 (AAPL), \$14 (MSFT), \$9 (ORCL)
- **Log-y axis** is essential here: on a linear axis NVDA's scale flattens the other three

WHAT THE PIPELINE SHOWS

- Each tool from today appears on one dataset, in a natural order
- The code is short because each tool does one job well and hands off cleanly
- A single swap of inputs (different tickers, different date window) produces a new analysis with no rewrite
- We moved between **panel** (full file), **time-series** (AAPL), **cross-section** (2020-03-16), and **wide** (for return maths) without leaving the same table
- Later weeks add more tools to the same loop

PRACTICE

EXERCISE

Task: use the Tech 4 panel to answer four questions.

1. Load the CSV with the correct date parser, then verify Date is datetime64
2. Compute each ticker's annualised total return over the full sample. Which ticker won?
3. For 2020-03-16 (the COVID-crash Monday), print a cross-section of all four tickers' total return. Which performed best?
4. For NVDA alone, compute 60-day rolling volatility and plot it. Identify the two highest-volatility periods

Hints:

- Annualised return: $(1 + \bar{r}_{\text{daily}})^{252} - 1$, where $\bar{r}$ is the mean daily total return
- Annualised vol: daily std $\times \sqrt{252}$

PRACTICE: COKE VS PEPSI

WEEK 1 PRACTICE: COKE VS PEPSI (1/2)

Dataset (data/week1_assignment_data.txt, tab-separated): **13,078** rows of CRSP daily data on Coca-Cola (KO) and PepsiCo (PEP), 2000-01-03 to 2025-12-31. Columns: DlyCalDt (date as integer YYYYMMDD), DlyPrct (USD price), DlyCap (\$thousands market cap), DlyRet (daily total return), DlyPrctVol (daily \$ volume).

Task 1 – Load and inspect. Read the TSV with `pd.read_csv(..., sep="\t")`. The date column arrives as an integer in YYYYMMDD form, so `parse_dates` alone will not work. Convert with `pd.to_datetime(df["DlyCalDt"], format="%Y%m%d")`. Save the result as `week1_assignment_data.parquet`. Print shape, date range, the unique tickers with their security names, and dtypes.

Task 2 – Stage 1 sanity check. Repeat the workshop's three checks on this fresh dataset: duplicate (Date, Ticker) rows (`.duplicated(...).sum()`), nulls per column (`.isna().sum()`), and equal per-ticker row counts (`.groupby("Ticker").size()`). State in one sentence whether the panel needs cleaning or is clean enough to use as is.

Task 3 – Per-ticker summary, two ways. Write one DuckDB SQL query against the Parquet that returns, per ticker: `mean_price`, `sd_price` (population SD via `STDDEV_POP`), `mean_cap_million_usd` (`AVG(DlyCap)/1000`), and `n_days`, sorted by `mean_price` descending. Reproduce the same DataFrame with a single pandas `groupby + agg` chain (use `lambda s: s.std(ddof=0)` for the population SD). Confirm that the two results agree.

WEEK 1 PRACTICE: COKE VS PEPSI (2/2)

Task 4 – Two slices. (a) Cross-section: filter to 16 March 2020 (the COVID-crash Monday). Print the four rows. Which company performed better that day, and by how many percentage points? (b) Time-series: filter to KO alone, sort by Date, print the first and last two rows. State the resulting shape (rows $\times$ columns).

Task 5 – KO vs PEP, \$1 invested. Pivot the panel to wide form on DlyRet (one column per ticker). Compute the *value of \$1* invested on 2000-01-03, held to 2025-12-31, using $V_t = \prod_{s \leq t} (1 + r_s)$ – not $V_t - 1$ (it goes negative during drawdowns and breaks the log axis). Print the final dollar value per ticker. Plot both paths on a single chart with a logarithmic y -axis, then save as `ko_vs_pep_growth.pdf`. In two or three sentences: which company won, by what factor, and one finance-relevant reason this is a sensible comparison (think about why one would compare KO and PEP rather than KO and AAPL).

Practice for Quiz 1 – not graded. You must be able to run all five tasks end-to-end. The Quiz 1 questions are similar to this pipeline.

Full brief in `ASSIGNMENT.md`.

SUMMARY

- The **Data Factory Floor** (DFF) has four stages with critical attributes (authenticity, utility, clarity, added value). Hand-offs between them are the most common source of bugs, caught by a data contract: an agreed schema, types, and update frequency pinned to each hand-off
- Financial data has four generator classes (fundamental, market, generated, alternative) and three structure classes (structured, semi-structured, unstructured)
- Pandas DataFrames hold tabular data as columns that each have a data type, plus row and column labels — each column is backed by a numpy array, or a pandas extension array for dtypes like `string` or `category`
- Parquet preserves types and compresses columns. Use it for yourself, CSV for hand-offs
- DuckDB runs SQL directly against Parquet or CSV files without a separate server or load step
- Each structured finance dataset is a **panel**, a **time-series**, or a **cross-section**. Panel is the superset, the other two are slices of it
- The Tech 4 workshop panel put all the tools above into one short pipeline

Next week: Building a Narrative from Data – turning a Stage 1 long table into a five-figure macro story with FT-style figure conventions.

REFERENCES I

- Apache Software Foundation. Apache Parquet file format specification. Online documentation, 2024. URL <https://parquet.apache.org/docs/file-format/>. Living specification maintained at <https://github.com/apache/parquet-format>.
- E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970. doi: 10.1145/362384.362685.
- Juraj Hric and Yiping Lin. *Applied Data Science in FinTech: Models, Tools, and Case Studies*. Routledge, London, 2026. ISBN 9781032762579. URL <https://www.routledge.com/Applied-Data-Science-in-FinTech-Models-Tools-and-Case-Studies/Hric-Lin/p/book/9781032762579>. Course-mandated textbook.
- Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. O'Reilly Media, 3 edition, 2022. ISBN 9781098104030. URL <https://wesmckinney.com/book/>.
- Mark Raasveldt and Hannes Mühleisen. DuckDB: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984. Association for Computing Machinery, 2019. doi: 10.1145/3299869.3320212. URL <https://doi.org/10.1145/3299869.3320212>.
- Brandon Rohrer. Data engineering for beginners, 2024. URL https://brandonrohrer.org/data_eng_for_beginners.html. Online article. Publication date not explicit on page; year assigned from access date.
- Hadley Wickham. Tidy data. *Journal of Statistical Software*, 59(10):1–23, 2014. doi: 10.18637/jss.v059.i10. URL <https://www.jstatsoft.org/v59/i10/>.